A5. Membership Numbers, Referral Codes & Weekly Check-in

Category B Assignment - Problem 5 (Intermediate to Advanced)

Approach

A shared static counter is incremented in the constructor and its resulting ID is stored in a final field. Referral codes are checked by character-by-character tests, not regex. The overloaded payment method delegates to payFee(int). Null check-in entries are skipped safely.

Java Solution

```
class GymMember {
    private static int enrolled = 0;
    private final String membershipNumber;
    protected final int monthlyFee;
    private int feesPaid;
    public GymMember(int monthlyFee) {
        if (monthlyFee <= 0) throw new IllegalArgumentException("Fee must be positive");
        this.monthlyFee = monthlyFee;
        enrolled++;
        membershipNumber = "GYM-" + (2000 + enrolled);
    }
    public String getMembershipNumber() { return membershipNumber; }
    public void payFee(int amount) {
        if (amount < 0) throw new IllegalArgumentException("Amount cannot be negative");
        feesPaid += amount;
    }
    public void payFee(int amount, String mode) {
        System.out.println("Payment mode: " + mode);
        payFee(amount); // reuse flat-amount method
    }
    public int getFeesPaid() { return feesPaid; }
    public static int getMembersEnrolled() { return enrolled; }
}
class GroupClassMember extends GymMember {
    private final String className;
    public GroupClassMember(int fee, String className) { super(fee); this.className=className; }
}
class MembershipBatch {
    static boolean isValidReferralCode(String code) {
        if (code == null || code.length() != 4) return false;
        return code.charAt(0)=='G' && Character.isDigit(code.charAt(1)) &&
               Character.isDigit(code.charAt(2)) && Character.isUpperCase(code.charAt(3));
    }
    static String processWeeklyCheckIn(GymMember[] members) {
        int processed=0, nullSkipped=0, group=0, individual=0;
        for (GymMember member : members) {
            if (member == null) { nullSkipped++; continue; }
            processed++;
            if (member instanceof GroupClassMember) group++; else individual++;
        }
        return processed + " processed | " + nullSkipped + " null skipped | " +
               group + " group | " + individual + " individual";
    }
    public static void main(String[] args) {
        GymMember m = new GymMember(1000);
        System.out.println(m.getMembershipNumber());
        System.out.println(GymMember.getMembersEnrolled());
        System.out.println(isValidReferralCode("G45B"));
        m.payFee(500); m.payFee(500, "UPI");
        System.out.println(m.getFeesPaid());
    }
}
```

Sample Output

```
GYM-2001
1
true
Payment mode: UPI
1000
```

Notes

The prompt example names a final membershipNumber field but also shows it as directly readable. This implementation keeps the field private and exposes a getter so it cannot be reassigned externally.